Date: 10/09/25 UE233074

PRACTICAL-5

Implementation of Lemmatization and Viterbi Algorithm

Lemmatization

Definition:
Lemmatization is a process in **Natural Language Processing (NLP)** that reduces a word to its **base or dictionary form (lemma)**. Unlike stemming (which simply chops off word endings), lemmatization considers the **morphological structure and grammar** of the word.

Example:

- *running*, *ran* → **run**
- *better* → **good**

How Lemmatization Works

- It looks at **Part-of-Speech (POS)** (noun, verb, adjective, adverb) of the word.
- Uses **vocabulary + morphological analysis** to find the correct lemma.
- Often relies on **lexicons** like WordNet.

Types of Lemmatization

1. **Rule-Based Lemmatization**
    - Uses linguistic rules (prefix/suffix rules, verb conjugation, pluralization rules).
    - Example:
        - "flies" (verb) → "fly"
        - "flies" (noun) → "fly"
    - Needs POS tagging to avoid ambiguity.
2. **Dictionary-Based Lemmatization**
    - Uses a predefined dictionary/lexicon mapping words to their lemmas.
    - Example: WordNet Lemmatizer in NLTK.
    - Accurate but depends on dictionary coverage.
3. **Machine Learning Based Lemmatization**
    - Combines **rules + dictionary** for better results.
    - Example: spaCy lemmatizer.

Difference from Stemming

- **Stemming:** "studies" → "studi" (just chopped)
- **Lemmatization:** "studies" → "study" (proper root word)

Date: 10/09/25 UE233074

Viterbi Algorithm -

Definition:
The Viterbi algorithm is a **dynamic programming algorithm** used to find the most likely sequence of hidden states (called **Viterbi path**) in a **Hidden Markov Model (HMM)**, given a sequence of observed events.

It is widely used in **speech recognition, POS tagging, machine translation, and bioinformatics**.

1. Transition Probability (aij) -

- **Definition:** The probability of moving from one hidden state si to another hidden state sj.
- **Meaning:**

$$a_{ij} = P(s_j \mid s_i)$$

If we are currently in state si at time t−1, this gives the likelihood of moving to state sj at time t.

2. Emission Probability (bj(ot))

- **Definition:** The probability of observing a symbol (word, sound, etc.) given the current hidden state.
- **Meaning:**

$$b_j(o_t) = P(o_t \mid s_j)$$

If the hidden state is sj, this gives the likelihood of generating observation ot.

CODE:-

1. Applying Rule-Based Lemmatization:

```
def simple_rule_lemmatizer(word):

    word = word.lower()
    # Rule 1: Remove '-ing' and add 'e' if applicable
    if word.endswith("ing"):
        return word[:-3] + 'e'
    # Rule 2: Remove '-ed'
    if word.endswith("ed"):
        return word[:-2]
    # Rule 3: Handle plurals ending in 's'
    if word.endswith("s"):
        # This is a very simple rule and won't work for all cases (e.g., "boxes")
        return word[:-1]
    # If no rule applies, return the original word
    return word

# Correctly handled by the rules
word1 = "walked"
print(f"Original word: '{word1}' -> Lemma (Rule-Based): '{simple_rule_lemmatizer(word1)}'")

word2 = "running" # Fails with this rule
print(f"Original word: '{word2}' -> Lemma (Rule-Based): '{simple_rule_lemmatizer(word2)}'")

word3 = "cats"
print(f"Original word: '{word3}' -> Lemma (Rule-Based): '{simple_rule_lemmatizer(word3)}'")

print("\n--- Example of a failure case ---")
```

Output:

```
# Fails because there's no rule for irregular verbs
word4 = "ran"
print(f"Original word: '{word4}' -> Lemma (Rule-Based): '{simple_rule_lemmatizer(word4)}'")
```

```
Original word: 'walked' -> Lemma (Rule-Based): 'walk'
Original word: 'running' -> Lemma (Rule-Based): 'runne'
Original word: 'cats' -> Lemma (Rule-Based): 'cat'

--- Example of a failure case ---
Original word: 'ran' -> Lemma (Rule-Based): 'ran'
```

2. Importing some libraries.

```
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
from nltk import pos_tag
from nltk.tokenize import word_tokenize
import nltk
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

3. Applying WordNet for Dictionary-Based Lemmatization:

```
# Create a lemmatizer instance
lemmatizer = WordNetLemmatizer()

def get_wordnet_pos(treebank_tag):

    if treebank_tag.startswith('J'):
        return wordnet.ADJ  # Adjective
    elif treebank_tag.startswith('V'):
        return wordnet.VERB # Verb
    elif treebank_tag.startswith('N'):
        return wordnet.NOUN # Noun
    elif treebank_tag.startswith('R'):
        return wordnet.ADV  # Adverb
    else:
        return wordnet.NOUN

# Example 1: Irregular verb
word1 = "run"
tokens = word_tokenize(word1)
pos_tagged_tokens = pos_tag(tokens)
pos_for_lemmatizer = get_wordnet_pos(pos_tagged_tokens[0][1])
lemma = lemmatizer.lemmatize(word1, pos=pos_for_lemmatizer)
print(f"Original word: '{word1}' -> Lemma (Dictionary-Based): '{lemma}'")
```

```
# Example 2: Irregular adjective
word2 = "better"
tokens = word_tokenize(word2)
pos_tagged_tokens = pos_tag(tokens)
pos_for_lemmatizer = get_wordnet_pos(pos_tagged_tokens[0][1])
lemma = lemmatizer.lemmatize(word2, pos=pos_for_lemmatizer)
print(f"Original word: '{word2}' -> Lemma (Dictionary-Based): '{lemma}'")

# Example 3: Regular verb without a POS tag (will fail)
word3 = "running"
lemma = lemmatizer.lemmatize(word3)
print(f"Original word: '{word3}' -> Lemma (Default Noun): '{lemma}'")

# Example 4: Regular verb with a POS tag (will succeed)
tokens = word_tokenize(word3)
pos_tagged_tokens = pos_tag(tokens)
pos_for_lemmatizer = get_wordnet_pos(pos_tagged_tokens[0][1])
lemma = lemmatizer.lemmatize(word3, pos=pos_for_lemmatizer)
print(f"Original word: '{word3}' -> Lemma (Verb POS): '{lemma}'")
```

```
Original word: 'run' -> Lemma (Dictionary-Based): 'run'
Original word: 'better' -> Lemma (Dictionary-Based): 'well'
Original word: 'running' -> Lemma (Default Noun): 'running'
Original word: 'running' -> Lemma (Verb POS): 'run'
```

4. Machine Learning Based Lemmatization –

```
import spacy

try:
    nlp = spacy.load("en_core_web_sm")
except OSError:
    print("Language model 'en_core_web_sm' not found.")
    print("Please run: python -m spacy download en_core_web_sm")
    exit()

def get_spacy_lemmas(text):
    doc = nlp(text)
    lemmas = [token.lemma_ for token in doc]
    return lemmas

# Example 1: Irregular verb
text1 = "I went home and ate an apple."
lemmas1 = get_spacy_lemmas(text1)
print(f"Original text: '{text1}'")
print(f"Lemmas (ML-Based): {lemmas1}")

# Example 2: Irregular adjective and a regular verb
text2 = "She is the better athlete, always running to win."
lemmas2 = get_spacy_lemmas(text2)
print(f"Original text: '{text2}'")
print(f"Lemmas (ML-Based): {lemmas2}")
```

```
# Example 2: Irregular adjective and a regular verb
text2 = "She is the better athlete, always running to win."
lemmas2 = get_spacy_lemmas(text2)
print(f"Original text: '{text2}'")
print(f"Lemmas (ML-Based): {lemmas2}")

# Example 3: A plural noun and a new/unseen word
text3 = "The children are blogs their lives."
lemmas3 = get_spacy_lemmas(text3)
print(f"Original text: '{text3}'")
print(f"Lemmas (ML-Based): {lemmas3}")
```

```
Original text: 'I went home and ate an apple.'
Lemmas (ML-Based): ['I', 'go', 'home', 'and', 'eat', 'an', 'apple', '.']
Original text: 'She is the better athlete, always running to win.'
Lemmas (ML-Based): ['she', 'be', 'the', 'well', 'athlete', ',', 'always', 'run', 'to', 'win', '.']
Original text: 'The children are blogs their lives.'
Lemmas (ML-Based): ['the', 'child', 'be', 'blog', 'their', 'life', '.']
```

5. Implementation of Viterbi Algorithm

```
import pandas as pd
import numpy as np

# Transition probabilities (from each state to observation)
states = ['start', 'DT', 'NN', 'VB']
observations = ['DT', 'NN', 'VB']
transition_probabilities = {
    'DT': [0.8, 0, 0, 0.5],
    'NN': [0.2, 0.9, 0.5, 0.5],
    'VB': [0, 0.1, 0.5, 0]
}
transition_df = pd.DataFrame(transition_probabilities, index=states)

# Emission probabilities for the sentence "The fans watch the race"
states = ['DT', 'NN', 'VB']
observations_sentence = ['The', 'fans', 'watch', 'the', 'race']
emission_probabilities_sentence = {
    'The': [0.2, 0.0, 0.0],
    'fans': [0.0, 0.1, 0.2],
    'watch': [0.0, 0.1, 0.3],
    'the': [0.25, 0.0, 0.0],   # lowercase 'the'
    'race': [0.0, 0.3, 0.15]
}
emission_df_sentence = pd.DataFrame(emission_probabilities_sentence, index=states)
```

```python
# Viterbi algorithm implementation
def viterbi(transition_df, emission_df, observations_sentence, states):
    n_states = len(states)
    n_obs = len(observations_sentence)

    # Initialize the probability matrix and path matrix
    viterbi_prob = np.zeros((n_states, n_obs))
    backpointer = np.zeros((n_states, n_obs), dtype=int)

    # Initialization step
    for s in range(n_states):
        trans_prob = transition_df.loc['start', observations[s]]
        emis_prob = emission_df.iloc[s, 0]  # first observation
        viterbi_prob[s, 0] = trans_prob * emis_prob
        backpointer[s, 0] = 0
    print("Initialization:")
    print(viterbi_prob[:, 0])

    # Recursion step
    for t in range(1, n_obs):
        for s in range(n_states):
            max_prob = 0
            max_state = 0
            for s_prev in range(n_states):
                trans_prob = transition_df.loc[states[s_prev], observations[s]]
                emis_prob = emission_df.iloc[s, t]
                prob = viterbi_prob[s_prev, t-1] * trans_prob * emis_prob
                if prob > max_prob:
                    max_prob = prob
                    max_state = s_prev
            viterbi_prob[s, t] = max_prob
            backpointer[s, t] = max_state
        print(f"Step {t}:")
        print(viterbi_prob[:, t])

    # Termination step
    best_path_prob = np.max(viterbi_prob[:, -1])
    best_last_state = np.argmax(viterbi_prob[:, -1])

    # Trace back the path
    best_path = [best_last_state]
    for t in range(n_obs - 1, 0, -1):
        best_last_state = backpointer[best_last_state, t]
        best_path.insert(0, best_last_state)

    best_path_states = [states[state] for state in best_path]

    return best_path_states, best_path_prob
```

```python
# Run the Viterbi algorithm
best_path_states, best_path_prob = viterbi(transition_df, emission_df_sentence, observations_sentence, states)

print("\nSentence -")
print("The fans watch the race.")
print("\nBest POS tag sequence:")
print(best_path_states)
print("Probability of the best sequence:")
print(best_path_prob)
```

```
Initialization:
[0.16 0.   0.  ]
Step 1:
[0.     0.0144 0.0032]
Step 2:
[0.      0.00072 0.00216]
Step 3:
[0.00027 0.      0.     ]
Step 4:
[0.00e+00 7.29e-05 4.05e-06]

Sentence -
The fans watch the race.

Best POS tag sequence:
['DT', 'NN', 'VB', 'DT', 'NN']
Probability of the best sequence:
7.290000000000001e-05
```